

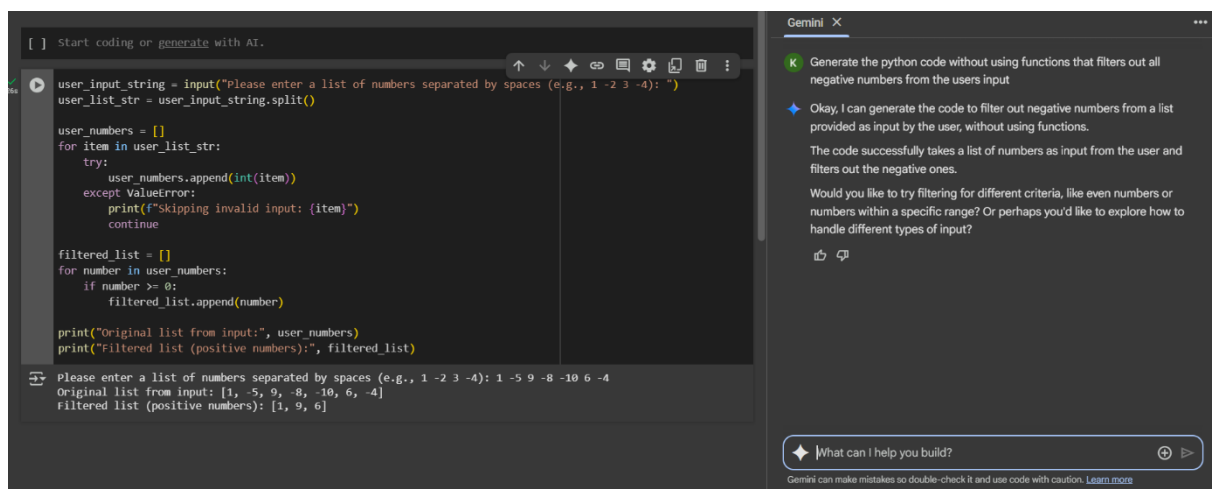
LAB– 2.2

Course_name:AI Assisted coding

Enrollment_no:2403A52002

Task -1: Description

- ❖: Use Gemini in colab to write a function that filter out all negative numbers from a given list
- ❖**Prompt:** Generate the python code without using functions that filters out all negative numbers from the users input.



The screenshot displays a Google Colab environment. On the left, a code editor contains a Python script that prompts the user for a list of numbers, processes the input, and prints the original and filtered (positive) lists. The output shows the original list [1, -5, 9, -8, -10, 6, -4] and the filtered list [1, 9, 6]. On the right, the Gemini AI chat interface is open, showing a conversation where the user asks for code to filter negative numbers without using functions, and Gemini provides a helpful response.

```
[ ] Start coding or generate with AI.
```

```
user_input_string = input("Please enter a list of numbers separated by spaces (e.g., 1 -2 3 -4): ")
user_list_str = user_input_string.split()

user_numbers = []
for item in user_list_str:
    try:
        user_numbers.append(int(item))
    except ValueError:
        print(f"Skipping invalid input: {item}")
        continue

filtered_list = []
for number in user_numbers:
    if number >= 0:
        filtered_list.append(number)

print("Original list from input:", user_numbers)
print("Filtered list (positive numbers):", filtered_list)
```

Please enter a list of numbers separated by spaces (e.g., 1 -2 3 -4): 1 -5 9 -8 -10 6 -4
Original list from input: [1, -5, 9, -8, -10, 6, -4]
Filtered list (positive numbers): [1, 9, 6]

Gemini X

Generate the python code without using functions that filters out all negative numbers from the users input

Okay, I can generate the code to filter out negative numbers from a list provided as input by the user, without using functions. The code successfully takes a list of numbers as input from the user and filters out the negative ones.

Would you like to try filtering for different criteria, like even numbers or numbers within a specific range? Or perhaps you'd like to explore how to handle different types of input?

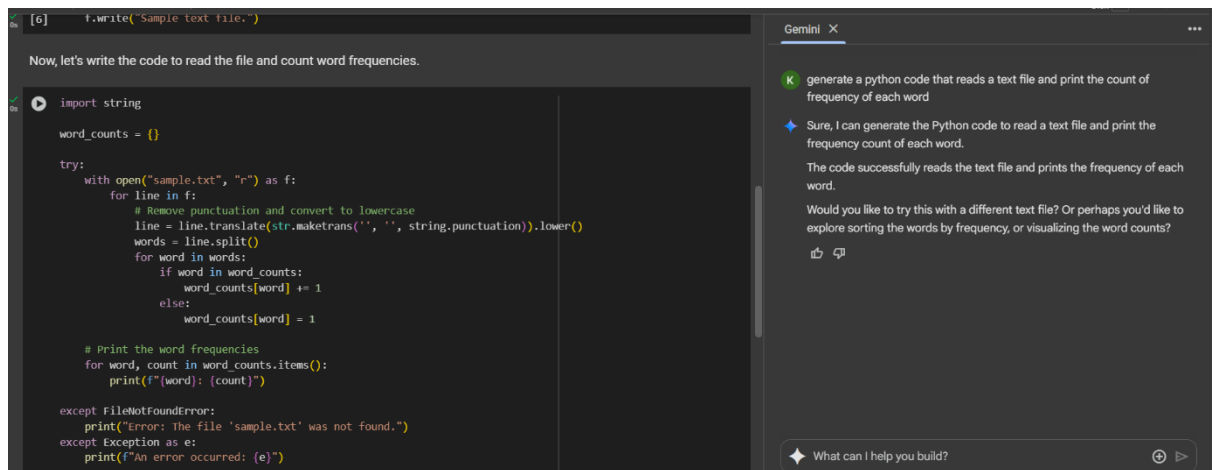
What can I help you build?

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

- ❖**Observation:** Gemini can understand the task very clearly. It takes very less time to complete the task and the output is accurate and matches the example provided.

Task-2:

- ❖ **Description:** Ask Gemini to generate code that reads a text file and counts the frequency of each word. Then ask it to explain the code.
- ❖ **Prompt:** Generate a python code that reads a text file and print the count of frequency of each word.



The screenshot shows a code editor on the left and a Gemini chat interface on the right. The code editor contains the following Python code:

```
[6] f.write("Sample text file.")

Now, let's write the code to read the file and count word frequencies.

import string

word_counts = {}

try:
    with open("sample.txt", "r") as f:
        for line in f:
            # Remove punctuation and convert to lowercase
            line = line.translate(str.maketrans('', '', string.punctuation)).lower()
            words = line.split()
            for word in words:
                if word in word_counts:
                    word_counts[word] += 1
                else:
                    word_counts[word] = 1

    # Print the word frequencies
    for word, count in word_counts.items():
        print(f"{word}: {count}")

except FileNotFoundError:
    print("Error: The file 'sample.txt' was not found.")
except Exception as e:
    print(f"An error occurred: {e}")
```

The Gemini chat interface on the right shows the following conversation:

Gemini ✕ ...

K generate a python code that reads a text file and print the count of frequency of each word

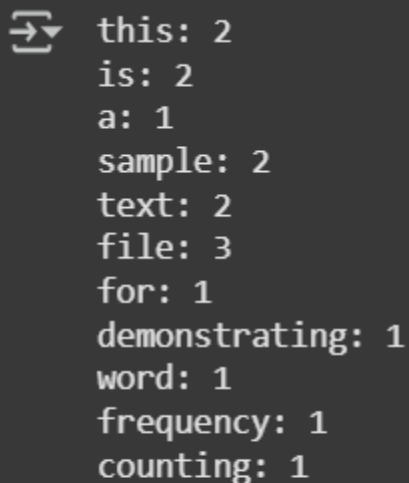
◆ Sure, I can generate the Python code to read a text file and print the frequency count of each word.

The code successfully reads the text file and prints the frequency of each word.

Would you like to try this with a different text file? Or perhaps you'd like to explore sorting the words by frequency, or visualizing the word counts?

👍 🔄

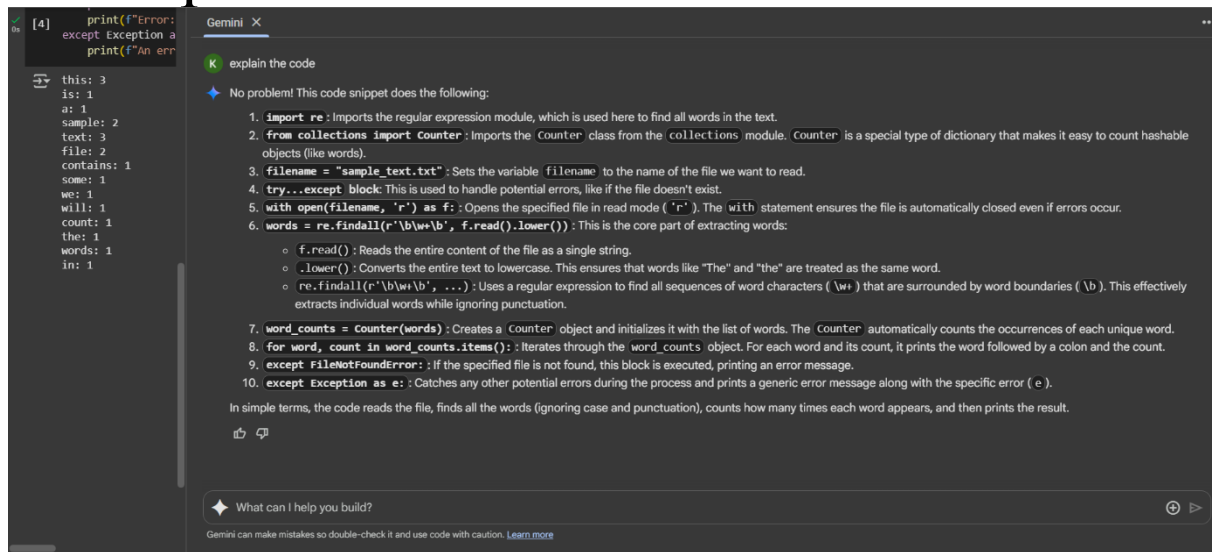
◆ What can I help you build? ⓘ ▶



The screenshot shows the output of the Python code, which is a list of words and their frequencies:

```
⇒ this: 2
is: 2
a: 1
sample: 2
text: 2
file: 3
for: 1
demonstrating: 1
word: 1
frequency: 1
counting: 1
```

❖ Explanation of code:



The screenshot shows a code editor with a Python script on the left and a Gemini AI chat window on the right. The script counts word frequencies in a file. The AI provides a detailed explanation of each line of code.

```
[a] print(f"Error: {e}")
except Exception as e:
    print(f"An error occurred: {e}")

this: 3
is: 1
a: 1
sample: 2
text: 3
file: 2
contains: 1
some: 1
we: 1
will: 1
count: 1
the: 1
words: 1
in: 1
```

Gemini explain the code

No problem! This code snippet does the following:

- `import re`: Imports the regular expression module, which is used here to find all words in the text.
- `from collections import Counter`: Imports the `Counter` class from the `collections` module. `Counter` is a special type of dictionary that makes it easy to count hashable objects (like words).
- `filename = "sample_text.txt"`: Sets the variable `filename` to the name of the file we want to read.
- `try...except` block: This is used to handle potential errors, like if the file doesn't exist.
- `with open(filename, 'r') as f:`: Opens the specified file in read mode (`'r'`). The `with` statement ensures the file is automatically closed even if errors occur.
- `words = re.findall(r'\b\w+\b', f.read().lower())`: This is the core part of extracting words:
 - `f.read()`: Reads the entire content of the file as a single string.
 - `.lower()`: Converts the entire text to lowercase. This ensures that words like "The" and "the" are treated as the same word.
 - `re.findall(r'\b\w+\b', ...)`: Uses a regular expression to find all sequences of word characters (`\w+`) that are surrounded by word boundaries (`\b`). This effectively extracts individual words while ignoring punctuation.
- `word_counts = Counter(words)`: Creates a `Counter` object and initializes it with the list of words. The `Counter` automatically counts the occurrences of each unique word.
- `for word, count in word_counts.items():`: Iterates through the `word_counts` object. For each word and its count, it prints the word followed by a colon and the count.
- `except FileNotFoundError:`: If the specified file is not found, this block is executed, printing an error message.
- `except Exception as e:`: Catches any other potential errors during the process and prints a generic error message along with the specific error (`e`).

In simple terms, the code reads the file, finds all the words (ignoring case and punctuation), counts how many times each word appears, and then prints the result.

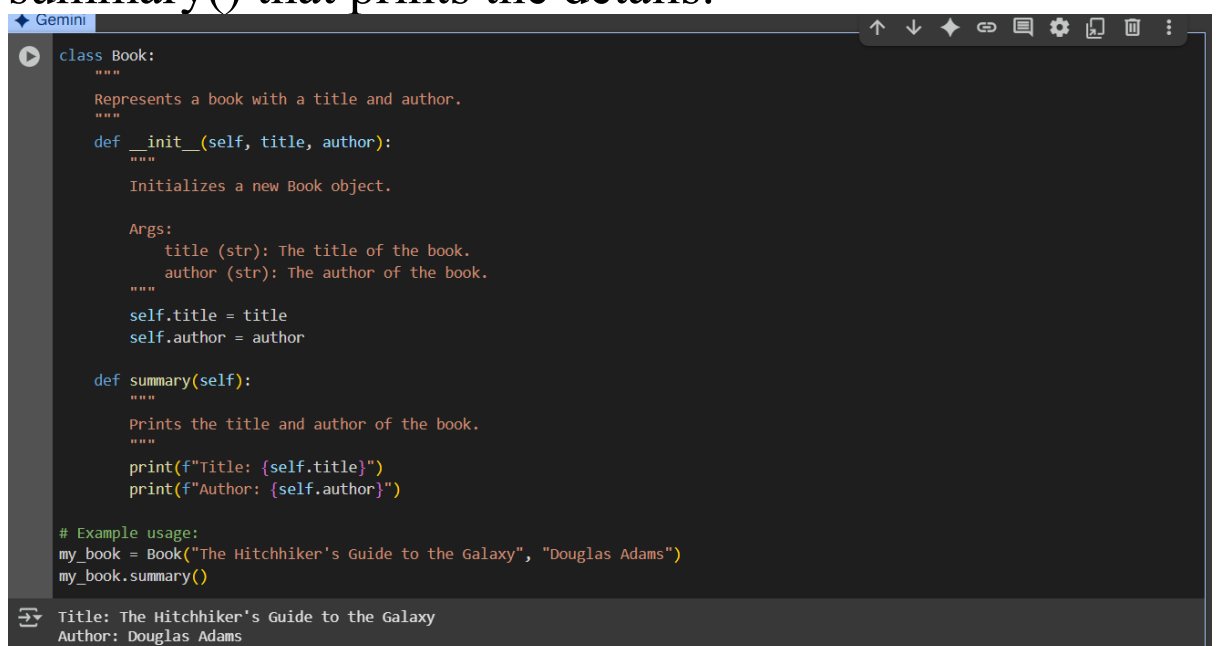
What can I help you build?

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

Task3:

❖ **Description:** Use cursor AI to generate a python class called Book with attributes, title, author, and a method `summary()` that prints the details.

❖ **Prompt:** Generate a python code of class called Book with attributes, title, author, and a method `summary()` that prints the details.



The screenshot shows a code editor with a Python class definition for a Book. The class has attributes `title` and `author`, and a method `summary()` that prints the details. An example usage is provided at the bottom.

```
class Book:
    """
    Represents a book with a title and author.
    """
    def __init__(self, title, author):
        """
        Initializes a new Book object.

        Args:
            title (str): The title of the book.
            author (str): The author of the book.
        """
        self.title = title
        self.author = author

    def summary(self):
        """
        Prints the title and author of the book.
        """
        print(f"Title: {self.title}")
        print(f"Author: {self.author}")

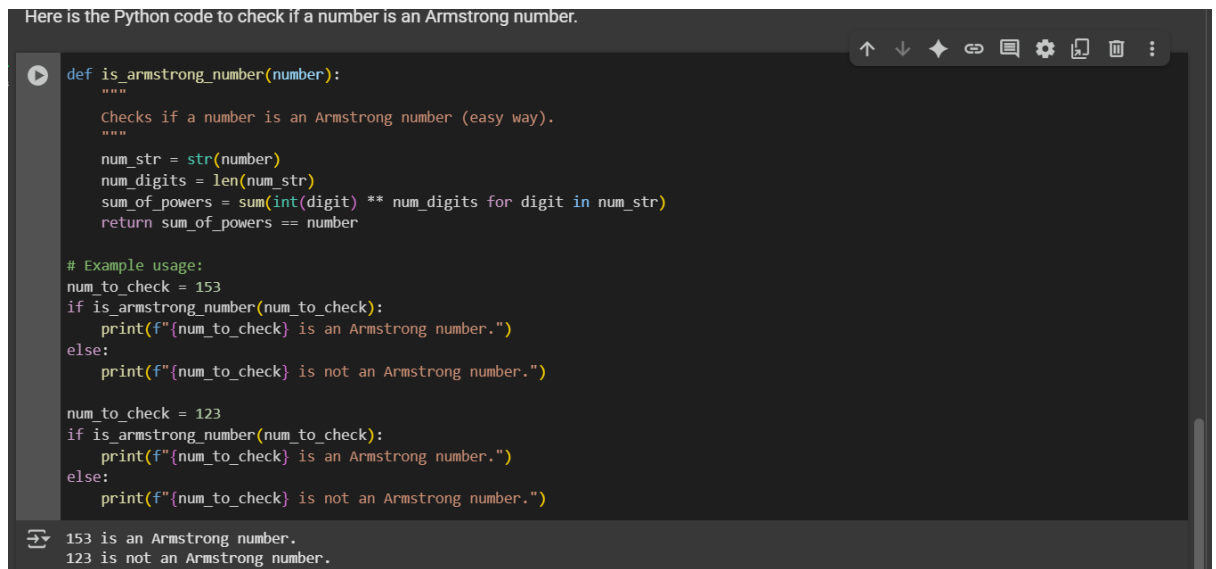
# Example usage:
my_book = Book("The Hitchhiker's Guide to the Galaxy", "Douglas Adams")
my_book.summary()
```

Title: The Hitchhiker's Guide to the Galaxy
Author: Douglas Adams

❖ **Observation:** Cursor AI correctly uses the class keyword to define the book class, following python object-oriented programming conventions. The `_init_()` method is implemented. The code assigns `self.title` and `self.author` properly.

Task4:

- ❖ **Description:** Ask the gemini to write a program that checks whether a number is an Armstrong number, and then modify it using Cursor AI to improve performance or structure.
- ❖ **Prompt:** Generate the python code to check whether a number is Armstrong number.

A screenshot of a code editor with a dark theme. At the top, a text label reads "Here is the Python code to check if a number is an Armstrong number." Below this, a Python function `def is_armstrong_number(number):` is defined. The function's docstring says "Checks if a number is an Armstrong number (easy way)." The code converts the number to a string, counts the digits, and calculates the sum of each digit raised to the power of the total number of digits. Below the function, there is an example usage section with two test cases: 153 and 123. The output at the bottom shows that 153 is an Armstrong number and 123 is not.

```
def is_armstrong_number(number):  
    """  
    Checks if a number is an Armstrong number (easy way).  
    """  
    num_str = str(number)  
    num_digits = len(num_str)  
    sum_of_powers = sum(int(digit) ** num_digits for digit in num_str)  
    return sum_of_powers == number  
  
# Example usage:  
num_to_check = 153  
if is_armstrong_number(num_to_check):  
    print(f"{num_to_check} is an Armstrong number.")  
else:  
    print(f"{num_to_check} is not an Armstrong number.")  
  
num_to_check = 123  
if is_armstrong_number(num_to_check):  
    print(f"{num_to_check} is an Armstrong number.")  
else:  
    print(f"{num_to_check} is not an Armstrong number.")
```

153 is an Armstrong number.
123 is not an Armstrong number.

❖ **Observation:** Gemini included sample input like 153 or 9474 and show that code correctly identifies them as Armstrong numbers. Raising each digit to the power of the number of digit.

Task5:

- ❖ **Description:** Use the both Gemini and Cursor AI to generate code for sorting a list of dictionaries by a specific key (e.g., age).
- ❖ **Prompt:** Generate a python code for sorting a list of dictionaries by a specific key
- ❖ **Gemini Code:**

```
Here is the Python code for sorting a list of dictionaries by a specific key.

# Sample list of dictionaries
list_of_dicts = [
    {'name': 'Alice', 'age': 30, 'city': 'New York'},
    {'name': 'Bob', 'age': 25, 'city': 'London'},
    {'name': 'Charlie', 'age': 35, 'city': 'Paris'},
    {'name': 'David', 'age': 25, 'city': 'Tokyo'}
]

# Specify the key to sort by
sort_key = 'age'

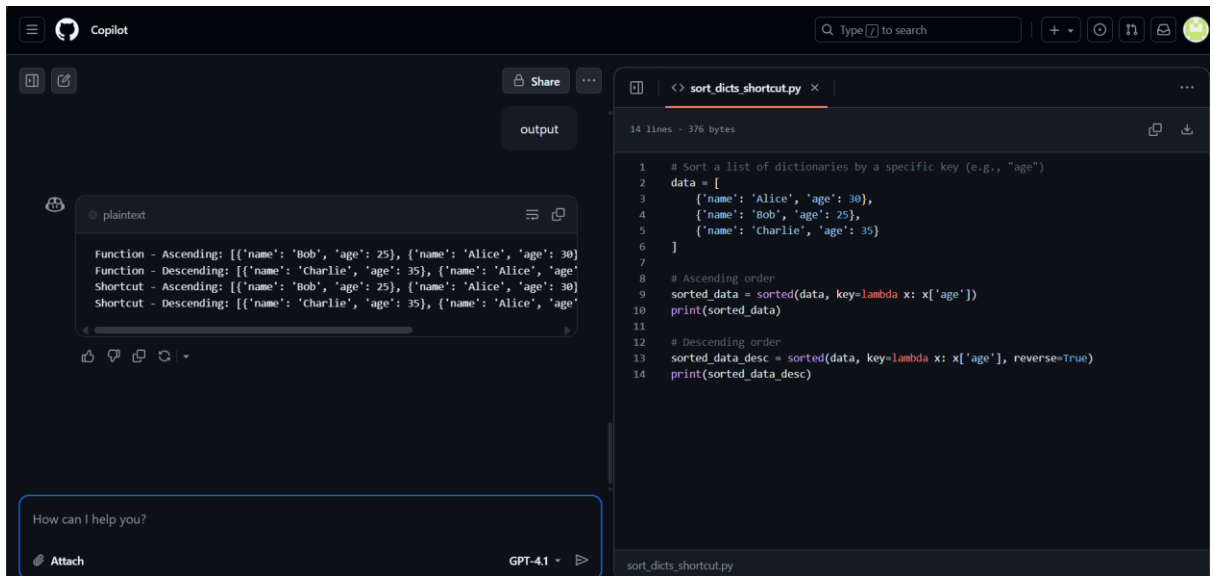
# Sort the list of dictionaries by the specified key
sorted_list = sorted(list_of_dicts, key=lambda x: x[sort_key])

# Print the sorted list
print(f"Sorted list by '{sort_key}':")
for item in sorted_list:
    print(item)

# You can also sort in reverse order by adding reverse=True
# sorted_list_reverse = sorted(list_of_dicts, key=lambda x: x[sort_key], reverse=True)
# print(f"\nSorted list by '{sort_key}' in reverse order:")
# for item in sorted_list_reverse:
#     print(item)

Sorted list by 'age':
{'name': 'Bob', 'age': 25, 'city': 'London'}
{'name': 'David', 'age': 25, 'city': 'Tokyo'}
{'name': 'Alice', 'age': 30, 'city': 'New York'}
{'name': 'Charlie', 'age': 35, 'city': 'Paris'}
```

❖ Cursor Code:



The screenshot shows the Cursor Code editor interface. On the left, there is a chat window with a "plaintext" tab. The chat history shows several messages from "Function" and "Shortcut" regarding sorting dictionaries by age in ascending and descending order. The main editor area displays a Python file named "sort_dicts_shortcut.py" with 14 lines of code. The code defines a list of dictionaries and uses the built-in `sorted()` function with a lambda key argument to sort them by age in both ascending and descending order. The output of the script is shown in the chat window.

```
1 # Sort a list of dictionaries by a specific key (e.g., "age")
2 data = [
3     {'name': 'Alice', 'age': 30},
4     {'name': 'Bob', 'age': 25},
5     {'name': 'Charlie', 'age': 35}
6 ]
7
8 # Ascending order
9 sorted_data = sorted(data, key=lambda x: x['age'])
10 print(sorted_data)
11
12 # Descending order
13 sorted_data_desc = sorted(data, key=lambda x: x['age'], reverse=True)
14 print(sorted_data_desc)
```

❖ **Observation:** Gemini typically uses python bulit-in `sorted()` function with a key argument like `lambda x:x['age']` , which is the standard and efficient way to sort.

Copilot sorts a list of dictionaries by the key "age".In ascending order: youngest to oldest.In descending order: oldest to youngest.Uses `sorted()` with a lambda to pick the "age" value.